

ПРИМЕЧАНИЕ

Возможность ложных пробуждений означает, что количество вызовов `pred` не определено. Мьютекс, ссылающийся на блокировку, будет заблокирован при каждом вызове `pred`, и выход из функции произойдет только при условии, что в результате вычисления `(bool)pred()` будет возвращено значение `true` или в результате вызова `Clock::now()` будет возвращено время, равное или большее времени, указанного с помощью `absolute_time`. Насчет продолжительности блокировки не дается никаких гарантий, только если функция вернула `false`, вызов `Clock::now()` возвращает время, равное или большее `absolute_time`, в том месте, где поток стал разблокированным.

Выдает

Любое исключение, выданное при вызове `pred`, или исключение `std::system_error`, если результат не может быть достигнут.

Синхронизация

Вызовы `notify_one()`, `notify_all()`, `wait()`, `wait_for()` и `wait_until()` в отношении одного и того же экземпляра `std::condition_variable` выстраиваются в последовательность. Вызов `notify_one()` или `notify_all()` разбудит только те потоки, которые приступили к ожиданию до этого вызова.

Г.3. Заголовок <atomic>

Заголовок <atomic> предоставляет набор базовых атомарных типов и операций над этими типами и набор класса для создания атомарных версий, определяемых пользователем типов, отвечающих определенным критериям.

Содержимое заголовка

```
#define ATOMIC_BOOL_LOCK_FREE см. описание
#define ATOMIC_CHAR_LOCK_FREE см. описание
#define ATOMIC_SHORT_LOCK_FREE см. описание
#define ATOMIC_INT_LOCK_FREE см. описание
#define ATOMIC_LONG_LOCK_FREE см. описание
#define ATOMIC_LLONG_LOCK_FREE см. описание
#define ATOMIC_CHAR16_T_LOCK_FREE см. описание
#define ATOMIC_CHAR32_T_LOCK_FREE см. описание
#define ATOMIC_WCHAR_T_LOCK_FREE см. описание
#define ATOMIC_POINTER_LOCK_FREE см. описание

#define ATOMIC_VAR_INIT(value) see description

namespace std
{
    enum memory_order;
    struct atomic_flag;
    typedef см. описание atomic_bool;
    typedef см. описание atomic_char;
    typedef см. описание atomic_char16_t;
    typedef см. описание atomic_char32_t;
    typedef см. описание atomic_schar;
```

```

typedef см. описание atomic_uchar;
typedef см. описание atomic_short;
typedef см. описание atomic_ushort;
typedef см. описание atomic_int;
typedef см. описание atomic_uint;
typedef см. описание atomic_long;
typedef см. описание atomic_ulong;
typedef см. описание atomic_llong;
typedef см. описание atomic_ullong;
typedef см. описание atomic_wchar_t;

typedef см. описание atomic_int_least8_t;
typedef см. описание atomic_uint_least8_t;
typedef см. описание atomic_int_least16_t;
typedef см. описание atomic_uint_least16_t;
typedef см. описание atomic_int_least32_t;
typedef см. описание atomic_uint_least32_t;
typedef см. описание atomic_int_least64_t;
typedef см. описание atomic_uint_least64_t;
typedef см. описание atomic_int_fast8_t;
typedef см. описание atomic_uint_fast8_t;
typedef см. описание atomic_int_fast16_t;
typedef см. описание atomic_uint_fast16_t;
typedef см. описание atomic_int_fast32_t;
typedef см. описание atomic_uint_fast32_t;
typedef см. описание atomic_int_fast64_t;
typedef см. описание atomic_uint_fast64_t;
typedef см. описание atomic_int8_t;
typedef см. описание atomic_uint8_t;
typedef см. описание atomic_int16_t;
typedef см. описание atomic_uint16_t;
typedef см. описание atomic_int32_t;
typedef см. описание atomic_uint32_t;
typedef см. описание atomic_int64_t;
typedef см. описание atomic_uint64_t;
typedef см. описание atomic_intptr_t;
typedef см. описание atomic_uintptr_t;
typedef см. описание atomic_size_t;
typedef см. описание atomic_ssize_t;
typedef см. описание atomic_ptrdiff_t;
typedef см. описание atomic_intmax_t;
typedef см. описание atomic_uintmax_t;

template<typename T>
struct atomic;

extern "C" void atomic_thread_fence(memory_order order);
extern "C" void atomic_signal_fence(memory_order order);

template<typename T>
T kill_dependency(T);
}

```

Г.3.1. Псевдонимы типа std::atomic_xxx

Для совместимости с предстоящим стандартом языка C предоставляются псевдонимы `typedef` для атомарных целочисленных типов. Для C++17 здесь должны быть псевдонимы `typedef` для соответствия специализации `std::atomic<T>`; для предыдущих стандартов C++ взамен этого они могут быть базовыми классами этой специализации с тем же интерфейсом (табл. Г.1).

Таблица Г.1. Атомарные псевдонимы `typedef` и соответствующие им специализации `std::atomic<>`

std::atomic_type	Специализация std::atomic<>
<code>std::atomic_char</code>	<code>std::atomic<char></code>
<code>std::atomic_schar</code>	<code>std::atomic<signed char></code>
<code>std::atomic_uchar</code>	<code>std::atomic<unsigned char></code>
<code>std::atomic_short</code>	<code>std::atomic<short></code>
<code>std::atomic_ushort</code>	<code>std::atomic<unsigned short></code>
<code>std::atomic_int</code>	<code>std::atomic<int></code>
<code>std::atomic_uint</code>	<code>std::atomic<unsigned int></code>
<code>std::atomic_long</code>	<code>std::atomic<long></code>
<code>std::atomic_ulong</code>	<code>std::atomic<unsigned long></code>
<code>std::atomic_llong</code>	<code>std::atomic<long long></code>
<code>std::atomic_ullong</code>	<code>std::atomic<unsigned long long></code>
<code>std::atomic_wchar_t</code>	<code>std::atomic<wchar_t></code>
<code>std::atomic_char16_t</code>	<code>std::atomic<char16_t></code>
<code>std::atomic_char32_t</code>	<code>std::atomic<char32_t></code>

Г.3.2. Макросы ATOMIC_xxx_LOCK_FREE

Эти макросы определяют, являются ли свободными от блокировки атомарные типы, соответствующие конкретным встроенным типам.

Объявления макросов

```
#define ATOMIC_BOOL_LOCK_FREE см. описание
#define ATOMIC_CHAR_LOCK_FREE см. описание
#define ATOMIC_SHORT_LOCK_FREE см. описание
#define ATOMIC_INT_LOCK_FREE см. описание
#define ATOMIC_LONG_LOCK_FREE см. описание
#define ATOMIC_LLONG_LOCK_FREE см. описание
#define ATOMIC_CHAR16_T_LOCK_FREE см. описание
#define ATOMIC_CHAR32_T_LOCK_FREE см. описание
#define ATOMIC_WCHAR_T_LOCK_FREE см. описание
#define ATOMIC_POINTER_LOCK_FREE см. описание
```

ATOMIC_xxx_LOCK_FREE может принимать значения 0, 1 или 2. Если значение равно 0, то операции как над знаковыми, так и над беззнаковыми атомарными типами, соответствующими названному типу, не обладают свободой от блокировок. Если значение равно 1, то операции для одних экземпляров этих типов могут быть, а для других не могут быть свободными от блокировок. И если значение равно 2, то эти операции всегда свободны от блокировок. Например, если значение ATOMIC_INT_LOCK_FREE равно 2, то операции над экземплярами `std::atomic<int>` и `std::atomic<unsigned>` всегда будут свободными от блокировок.

Макрос ATOMIC_POINTER_LOCK_FREE дает описание свойству свободы от блокировок тех операций, которые выполняются над специализациями атомарного указателя `std::atomic<T*>`.

Г.3.3. Макрос ATOMIC_VAR_INIT

Макрос ATOMIC_VAR_INIT предоставляет средство инициализации атомарной переменной конкретным значением.

Объявление

```
#define ATOMIC_VAR_INIT(value) см. описание
```

Макрос расширяется до последовательности лексем, которой можно воспользоваться для инициализации одного из стандартных атомарных типов конкретным значением в выражении такого вида:

```
std::atomic<type> x = ATOMIC_VAR_INIT(val);
```

Указанное значение должно быть совместимо с неатомарным типом, соответствующим атомарной переменной. Например:

```
std::atomic<int> i = ATOMIC_VAR_INIT(42);
std::string s;
std::atomic<std::string*> p = ATOMIC_VAR_INIT(&s);
```

Такая инициализация не является атомарной, и любое обращение к уже инициализируемой переменной со стороны другого потока, при котором инициализация не происходит до этого обращения, приводит к гонке за данными, а следовательно, и к неопределенному поведению.

Г.3.4. Перечисление std::memory_order

Перечисление `std::memory_order` используется для задания ограничений на порядок доступа к памяти при выполнении атомарных операций.

Объявление

```
typedef enum memory_order
{
    memory_order_relaxed, memory_order_consume,
    memory_order_acquire, memory_order_release,
    memory_order_acq_rel, memory_order_seq_cst
} memory_order;
```


Операции, помеченные различными значениями порядка доступа к памяти, ведут себя следующим образом (подробное описание ограничений доступа к памяти дано в главе 5).

`std::memory_order_relaxed`

Операция не обеспечивает каких-либо дополнительных ограничений порядка доступа к памяти.

`std::memory_order_release`

Операция относится к операциям освобождения конкретного места в памяти. Поэтому она синхронизируется с операцией захвата того же места в памяти для чтения сохраненного значения.

`std::memory_order_acquire`

Операция относится к операциям захвата конкретного места в памяти. Если сохраненное значение было записано операцией освобождения, то это сохранение синхронизируется с данной операцией.

`std::memory_order_acq_rel`

Эта операция должна относиться к операциям чтения — изменения — записи, и она ведет себя, как будто в отношении конкретного места в памяти задан порядок доступа как `std::memory_order_acquire`, так и `std::memory_order_release`.

`std::memory_order_seq_cst`

Эта операция формирует часть единого глобального порядка последовательно согласованных операций. Кроме того, если это операция сохранения, то она ведет себя как операция с семантикой `std::memory_order_release`; если это операция загрузки, то она ведет себя как операция с семантикой `std::memory_order_acquire`.

Если это операция чтения — изменения — записи, то она ведет себя и как операция с пометкой `std::memory_order_acquire`, и как операция с пометкой `std::memory_order_release`. Эта семантика является исходной для всех операций.

`std::memory_order_consume`

Эта операция относится к операциям потребления конкретного места в памяти. В стандарте C++17 утверждается, что это ограничение порядка доступа к памяти применяться не должно.

Г.3.5. Функция `std::atomic_thread_fence`

Функция `std::atomic_thread_fence()` вставляет в код «барьер» для принудительной установки ограничений порядка доступа к памяти между операциями.

Объявление

```
extern "C" void atomic_thread_fence(std::memory_order order);
```

Результат

Вставляет барьер с требуемыми ограничениями порядка доступа к памяти.

Барьер со значением параметра `order`, равным `std::memory_order_release`, `std::memory_order_acq_rel` или `std::memory_order_seq_cst`, синхронизируется с операцией захвата того же места в памяти, если эта операция захвата считывает значение, сохраненное атомарной операцией, следующей за барьером в том же потоке, в котором поставлен барьер.

Операция освобождения синхронизируется с барьером со значением параметра `order`, равным `std::memory_order_acquire`, `std::memory_order_acq_rel` или `std::memory_order_seq_cst`, если эта операция освобождения сохраняет значение, считываемое атомарной операцией, предшествующей барьеру в том же потоке, в котором поставлен барьер.

Выдает

Ничего не выдает.

Г.3.6. Функция `std::atomic_signal_fence`

Функция `std::atomic_signal_fence()` вставляет в код «барьер» для принудительной установки ограничений порядка доступа к памяти между операциями потока и операциями в обработке сигнала этого же потока.

Объявление

```
extern "C" void atomic_signal_fence(std::memory_order order);
```

Результат

Вставляет «барьер» с требуемыми ограничениями порядка доступа к памяти. Эквивалентна функции `std::atomic_thread_fence(order)`, за исключением того, что ограничения применяются только между потоком и обработчиком сигнала в том же самом потоке.

Выдает

Ничего не выдает.

Г.3.7. Класс `std::atomic_flag`

Класс `std::atomic_flag` предоставляет самый простой атомарный флаг. Это единственный тип данных, гарантированно свободный от блокировок согласно стандарту C++11 (хотя в большинстве реализации свободными от блокировок будут многие атомарные типы).

Экземпляр `std::atomic_flag` является либо *установленным*, либо *сброшенным*.

Определение класса

```
struct atomic_flag
{
```

```

atomic_flag() noexcept = default;
atomic_flag(const atomic_flag&) = delete;
atomic_flag& operator=(const atomic_flag&) = delete;
atomic_flag& operator=(const atomic_flag&) volatile = delete;

bool test_and_set(memory_order = memory_order_seq_cst) volatile
    noexcept;
bool test_and_set(memory_order = memory_order_seq_cst) noexcept;
void clear(memory_order = memory_order_seq_cst) volatile noexcept;
void clear(memory_order = memory_order_seq_cst) noexcept;
};
bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
bool atomic_flag_test_and_set(atomic_flag*) noexcept;
bool atomic_flag_test_and_set_explicit(
    volatile atomic_flag*, memory_order) noexcept;
bool atomic_flag_test_and_set_explicit(
    atomic_flag*, memory_order) noexcept;
void atomic_flag_clear(volatile atomic_flag*) noexcept;
void atomic_flag_clear(atomic_flag*) noexcept;
void atomic_flag_clear_explicit(
    volatile atomic_flag*, memory_order) noexcept;
void atomic_flag_clear_explicit(
    atomic_flag*, memory_order) noexcept;

#define ATOMIC_FLAG_INIT неопределено

```

Конструктор по умолчанию std::atomic_flag

Состояние установки или сброса созданного конструктором по умолчанию экземпляра `std::atomic_flag` не определяется. Для объектов со статической продолжительностью хранения инициализация должна быть статической.

Объявление

```
std::atomic_flag() noexcept = default;
```

Результат

Создает новый объект `std::atomic_flag` в неопределенном состоянии.

Выдает

Ничего не выдает.

Инициализация std::atomic_flag с помощью макроса ATOMIC_FLAG_INIT

Экземпляр `std::atomic_flag` можно проинициализировать с помощью макроса `ATOMIC_FLAG_INIT`, в таком случае он инициализируется в сброшенном состоянии. Для объектов со статической продолжительностью хранения инициализация должна быть статической.

Объявление

```
#define ATOMIC_FLAG_INIT unspecified
```

Использование

```
std::atomic_flag flag=ATOMIC_FLAG_INIT;
```

Результат

Создает новый объект `std::atomic_flag` в сброшенном состоянии.

Выдает

Ничего не выдает.

Компонентная функция `std::atomic_flag::test_and_set`

Атомарно устанавливает флаг и проверяет, был ли он установлен.

Объявление

```
bool test_and_set(memory_order order = memory_order_seq_cst) volatile
    noexcept;
bool test_and_set(memory_order order = memory_order_seq_cst) noexcept;
```

Результат

Атомарно устанавливает флаг.

Возвращает

`true`, если флаг был установлен в месте вызова, или `false`, если флаг был сброшен.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это атомарная операция чтения — изменения — записи места в памяти, содержащего `*this`.

Некомпонентная функция `std::atomic_flag_test_and_set`

Атомарно устанавливает флаг и проверяет, был ли он установлен.

Объявление

```
bool atomic_flag_test_and_set(volatile atomic_flag* flag) noexcept;
bool atomic_flag_test_and_set(atomic_flag* flag) noexcept;
```

Результат

```
return flag->test_and_set();
```

Некомпонентная функция `std::atomic_flag_test_and_set_explicit`

Атомарно устанавливает флаг и проверяет, был ли он установлен.

Объявление

```
bool atomic_flag_test_and_set_explicit(
    volatile atomic_flag* flag, memory_order order) noexcept;
```

```
bool atomic_flag_test_and_set_explicit(
    atomic_flag* flag, memory_order order) noexcept;
```

Результат

```
return flag->test_and_set(order);
```

Компонентная функция std::atomic_flag::clear

Атомарно сбрасывает флаг.

Объявление

```
void clear(memory_order order = memory_order_seq_cst) volatile noexcept;
void clear(memory_order order = memory_order_seq_cst) noexcept;
```

Предусловия

Предоставляемый параметр `order` должен иметь одно из следующих значений: `std::memory_order_relaxed`, `std::memory_order_release` или `std::memory_order_seq_cst`.

Результат

Атомарно сбрасывает флаг.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это атомарная операция сохранения места в памяти, содержащего `*this`.

Некомпонентная функция std::atomic_flag_clear

Атомарно сбрасывает флаг.

Объявление

```
void atomic_flag_clear(volatile atomic_flag* flag) noexcept;
void atomic_flag_clear(atomic_flag* flag) noexcept;
```

Результат

```
flag->clear();
```

Некомпонентная функция std::atomic_flag_clear_explicit

Атомарно сбрасывает флаг.

Объявление

```
void atomic_flag_clear_explicit(
    volatile atomic_flag* flag, memory_order order) noexcept;
void atomic_flag_clear_explicit(
    atomic_flag* flag, memory_order order) noexcept;
```

Результат

```
return flag->clear(order);
```

Г.3.8. Шаблон класса `std::atomic`

Классе `std::atomic` предоставляет оболочку с атомарными операциями для любого типа, удовлетворяющего следующим требованиям.

Параметр шаблона `BaseType` должен:

- ❑ иметь стандартный конструктор по умолчанию;
- ❑ иметь стандартный копирующий оператор присваивания;
- ❑ иметь стандартный деструктор;
- ❑ допускать поразрядное сравнение на равенство.

Это означает допустимость объектов `std::atomic<some-built-in-type>`, а также объектов `std::atomic<some-simple-struct>` и недопустимость объектов, подобных `std::atomic<std::string>`.

Кроме основного шаблона, имеются специализации для встроенных целочисленных типов и указателей для предоставления дополнительных операций, например `x++`.

Экземпляры `std::atomic` не являются `CopyConstructible` или `CopyAssignable`, поскольку соответствующие им операции не могут выполняться в рамках единой атомарной операции.

Определение класса

```
template<typename BaseType>
struct atomic
{
    using value_type = T;
    static constexpr bool is_always_lock_free = implementation-defined;
    atomic() noexcept = default;
    constexpr atomic(BaseType) noexcept;
    BaseType operator=(BaseType) volatile noexcept;
    BaseType operator=(BaseType) noexcept;

    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(BaseType, memory_order = memory_order_seq_cst)
        volatile noexcept;
    void store(BaseType, memory_order = memory_order_seq_cst) noexcept;
    BaseType load(memory_order = memory_order_seq_cst)
        const volatile noexcept;
    BaseType load(memory_order = memory_order_seq_cst) const noexcept;
    BaseType exchange(BaseType, memory_order = memory_order_seq_cst)
        volatile noexcept;
    BaseType exchange(BaseType, memory_order = memory_order_seq_cst)
        noexcept;

    bool compare_exchange_strong(
        BaseType & old_value, BaseType new_value,
```



```

        memory_order order = memory_order_seq_cst) volatile noexcept;
bool compare_exchange_strong(
    BaseType & old_value, BaseType new_value,
    memory_order order = memory_order_seq_cst) noexcept;
bool compare_exchange_strong(
    BaseType & old_value, BaseType new_value,
    memory_order success_order,
    memory_order failure_order) volatile noexcept;
bool compare_exchange_strong(
    BaseType & old_value, BaseType new_value,
    memory_order success_order,
    memory_order failure_order) noexcept;
bool compare_exchange_weak(
    BaseType & old_value, BaseType new_value,
    memory_order order = memory_order_seq_cst)
    volatile noexcept;
bool compare_exchange_weak(
    BaseType & old_value, BaseType new_value,
    memory_order order = memory_order_seq_cst) noexcept;
bool compare_exchange_weak(
    BaseType & old_value, BaseType new_value,
    memory_order success_order,
    memory_order failure_order) volatile noexcept;
bool compare_exchange_weak(
    BaseType & old_value, BaseType new_value,
    memory_order success_order,
    memory_order failure_order) noexcept;

operator BaseType () const volatile noexcept;
operator BaseType () const noexcept;
};

template<typename BaseType>
bool atomic_is_lock_free(volatile const atomic<BaseType>*) noexcept;
template<typename BaseType>
bool atomic_is_lock_free(const atomic<BaseType>*) noexcept;
template<typename BaseType>
void atomic_init(volatile atomic<BaseType>*, void*) noexcept;
template<typename BaseType>
void atomic_init(atomic<BaseType>*, void*) noexcept;
template<typename BaseType>
BaseType atomic_exchange(volatile atomic<BaseType>*, memory_order)
    noexcept;
template<typename BaseType>
BaseType atomic_exchange(atomic<BaseType>*, memory_order) noexcept;
template<typename BaseType>
BaseType atomic_exchange_explicit(
    volatile atomic<BaseType>*, memory_order) noexcept;
template<typename BaseType>
BaseType atomic_exchange_explicit(
    atomic<BaseType>*, memory_order) noexcept;
template<typename BaseType>
void atomic_store(volatile atomic<BaseType>*, BaseType) noexcept;

```



```

template<typename BaseType>
void atomic_store(atomic<BaseType>*, BaseType) noexcept;
template<typename BaseType>
void atomic_store_explicit(
    volatile atomic<BaseType>*, BaseType, memory_order) noexcept;
template<typename BaseType>
void atomic_store_explicit(
    atomic<BaseType>*, BaseType, memory_order) noexcept;
template<typename BaseType>
BaseType atomic_load(volatile const atomic<BaseType>*) noexcept;
template<typename BaseType>
BaseType atomic_load(const atomic<BaseType>*) noexcept;
template<typename BaseType>
BaseType atomic_load_explicit(
    volatile const atomic<BaseType>*, memory_order) noexcept;
template<typename BaseType>
BaseType atomic_load_explicit(
    const atomic<BaseType>*, memory_order) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_strong(
    volatile atomic<BaseType>*, BaseType * old_value,
    BaseType new_value) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_strong(
    atomic<BaseType>*, BaseType * old_value,
    BaseType new_value) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_strong_explicit(
    volatile atomic<BaseType>*, BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_strong_explicit(
    atomic<BaseType>*, BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_weak(
    volatile atomic<BaseType>*, BaseType * old_value, BaseType new_value)
    noexcept;
template<typename BaseType>
bool atomic_compare_exchange_weak(
    atomic<BaseType>*, BaseType * old_value, BaseType new_value) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_weak_explicit(
    volatile atomic<BaseType>*, BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_weak_explicit(
    atomic<BaseType>*, BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;

```

ПРИМЕЧАНИЕ

Хотя некомпонентные функции определены как шаблоны, они могут предоставляться в виде набора переопределяемых функций, и явная спецификация аргументов шаблона применяться не должна.

Конструктор по умолчанию `std::atomic`

Создает экземпляр `std::atomic` со значением инициализации по умолчанию.

Объявление

```
atomic() noexcept;
```

Результат

Создает новый объект `std::atomic` со значением инициализации по умолчанию. Для объектов со статической продолжительностью хранения эта инициализация является статической.

ПРИМЕЧАНИЕ

При инициализации конструктором по умолчанию экземпляров `std::atomic` с нестатической продолжительностью хранения рассчитывать на предсказуемость значения не получится.

Выдает

Ничего не выдает.

Некомпонентная функция `std::atomic_init`

Неатомарно сохраняет предоставленное значение в экземпляре `std::atomic<BaseType>`.

Объявление

```
template<typename BaseType>
void atomic_init(atomic<BaseType> volatile* p, BaseType v) noexcept;
template<typename BaseType>
void atomic_init(atomic<BaseType>* p, BaseType v) noexcept;
```

Результат

Неатомарно сохраняет значение `v` в `*p`. Вызов `atomic_init()` в отношении экземпляра `atomic<BaseType>`, который не был создан исходным образом или над которым с момента создания уже были проведены какие-либо операции, приводит к неопределенному поведению.

ПРИМЕЧАНИЕ

Поскольку сохранение носит неатомарный характер, любое конкурентное обращение к объекту, указанному с помощью `p` из другого потока (даже с применением атомарных операций), приводит к гонке за данными.

Выдает

Ничего не выдает.

Конструктор преобразования std::atomic

Создает экземпляр `std::atomic` с предоставленным значением `BaseType`.

Объявление

```
constexpr atomic(BaseType b) noexcept;
```

Результат

Создает новый объект `std::atomic` со значением `b`. Для объектов со статической продолжительностью хранения инициализация является статической.

Выдает

Ничего не выдает.

Оператор преобразования и присваивания std::atomic

Сохраняет новое значение в `*this`.

Объявление

```
BaseType operator=(BaseType b) volatile noexcept;
BaseType operator=(BaseType b) noexcept;
```

Результат

```
return this->store(b);
```

Компонентная функция std::atomic::is_lock_free

Определяет, являются ли операции над `*this` свободными от блокировок.

Объявление

```
bool is_lock_free() const volatile noexcept;
bool is_lock_free() const noexcept;
```

Возвращает

`true`, если операции над `*this` являются свободными от блокировок, или `false` в противном случае.

Выдает

Ничего не выдает.

Некомпонентная функция std::atomic_is_lock_free

Определяет, являются ли операции над `*this` свободными от блокировок.

Объявление

```
template<typename BaseType>
bool atomic_is_lock_free(volatile const atomic<BaseType>* p) noexcept;
template<typename BaseType>
bool atomic_is_lock_free(const atomic<BaseType>* p) noexcept;
```

Результат

```
return p->is_lock_free();
```

Статический элемент данных `std::atomic::is_always_lock_free`

Определяет, всегда ли операции над всеми объектами данного типа являются свободными от блокировок.

Объявление

```
static constexpr bool is_always_lock_free() = implementation-defined;
```

Значение

`true`, если операции над объектами данного типа всегда являются свободными от блокировок, или `false` в противном случае.

Компонентная функция `std::atomic::load`

Атомарно загружает текущее значение экземпляра `std::atomic`.

Объявление

```
BaseType load(memory_order order = memory_order_seq_cst)
const volatile noexcept;
BaseType load(memory_order order = memory_order_seq_cst) const noexcept;
```

Предусловия

Предоставляемый параметр `order` должен иметь одно из следующих значений: `std::memory_order_relaxed`, `std::memory_order_acquire`, `std::memory_order_consume` или `std::memory_order_seq_cst`.

Результат

Атомарно загружает значение, сохраненное в `*this`.

Возвращает

Значение, сохраненное в `*this` в месте вызова.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это атомарная операция загрузки места в памяти, содержащего `*this`.

Некомпонентная функция `std::atomic_load`

Атомарно загружает текущее значение экземпляра `std::atomic`.

Объявление

```
template<typename BaseType>
BaseType atomic_load(volatile const atomic<BaseType>* p) noexcept;
template<typename BaseType>
BaseType atomic_load(const atomic<BaseType>* p) noexcept;
```

Результат

```
return p->load();
```

Некомпонентная функция `std::atomic_load_explicit`

Атомарно загружает текущее значение экземпляра `std::atomic`.

Объявление

```
template<typename BaseType>
BaseType atomic_load_explicit(
    volatile const atomic<BaseType>* p, memory_order order) noexcept;
template<typename BaseType>
BaseType atomic_load_explicit(
    const atomic<BaseType>* p, memory_order order) noexcept;
```

Результат

```
return p->load(order);
```

Оператор преобразования в тип `BaseType` — `std::atomic::operator`

Загружает значение, сохраненное в `*this`.

Объявление

```
operator BaseType() const volatile noexcept;
operator BaseType() const noexcept;
```

Результат

```
return this->load();
```

Компонентная функция `std::atomic::store`

Атомарно сохраняет новое значение в экземпляре `atomic<BaseType>`.

Объявление

```
void store(BaseType new_value, memory_order order = memory_order_seq_cst)
    volatile noexcept;
void store(BaseType new_value, memory_order order = memory_order_seq_cst)
    noexcept;
```

Предусловия

Предоставляемый параметр `order` должен иметь одно из следующих значений: `std::memory_order_relaxed`, `std::memory_order_release` или `std::memory_order_seq_cst`.

Результат

Атомарно сохраняет значение `new_value` в `*this`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это атомарная операция сохранения места в памяти, содержащего `*this`.

Некомпонентная функция std::atomic_store

Атомарно сохраняет новое значение в экземпляре `atomic<BaseType>`.

Объявление

```
template<typename BaseType>
void atomic_store(volatile atomic<BaseType>* p, BaseType new_value)
    noexcept;
template<typename BaseType>
void atomic_store(atomic<BaseType>* p, BaseType new_value) noexcept;
```

Результат

```
p->store(new_value);
```

Некомпонентная функция std::atomic_store_explicit

Атомарно сохраняет новое значение в экземпляре `atomic<BaseType>`.

Объявление

```
template<typename BaseType>
void atomic_store_explicit(
    volatile atomic<BaseType>* p, BaseType new_value, memory_order order)
    noexcept;
template<typename BaseType>
void atomic_store_explicit(
    atomic<BaseType>* p, BaseType new_value, memory_order order) noexcept;
```

Результат

```
p->store(new_value, order);
```

Компонентная функция std::atomic::exchange

Атомарно сохраняет новое значение и считывает старое.

Объявление

```
BaseType exchange(
    BaseType new_value,
    memory_order order = memory_order_seq_cst)
    volatile noexcept;
```

Результат

Атомарно сохраняет `new_value` в `*this` и извлекает имеющееся значение `*this`.

Возвращает

Значение `*this`, существовавшее непосредственно перед сохранением.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это атомарная операция чтения — изменения — записи в отношении места в памяти, содержащего `*this`.

Некомпонентная функция `std::atomic_exchange`

Атомарно сохраняет новое значение в `atomic<BaseType>` и считывает старое.

Объявление

```
template<typename BaseType>
BaseType atomic_exchange(volatile atomic<BaseType>* p, BaseType new_value)
    noexcept;
template<typename BaseType>
BaseType atomic_exchange(atomic<BaseType>* p, BaseType new_value) noexcept;
```

Результат

```
return p->exchange(new_value);
```

Некомпонентная функция `std::atomic_exchange_explicit`

Атомарно сохраняет новое значение в `atomic<BaseType>` и считывает старое.

Объявление

```
template<typename BaseType>
BaseType atomic_exchange_explicit(
    volatile atomic<BaseType>* p, BaseType new_value, memory_order order)
    noexcept;
template<typename BaseType>
BaseType atomic_exchange_explicit(
    atomic<BaseType>* p, BaseType new_value, memory_order order) noexcept;
```

Результат

```
return p->exchange(new_value,order);
```

Компонентная функция `std::atomic::compare_exchange_strong`

Атомарно сравнивает значение с ожидаемым значением и сохраняет новое значение, если они равны друг другу. Если значения не равны, обновляет ожидаемое значение считанным значением.

Объявление

```
bool compare_exchange_strong(
    BaseType& expected, BaseType new_value,
    memory_order order = std::memory_order_seq_cst) volatile noexcept;
bool compare_exchange_strong(
    BaseType& expected, BaseType new_value,
    memory_order order = std::memory_order_seq_cst) noexcept;
bool compare_exchange_strong(
    BaseType& expected, BaseType new_value,
    memory_order success_order, memory_order failure_order)
    volatile noexcept;
bool compare_exchange_strong(
    BaseType& expected, BaseType new_value,
    memory_order success_order, memory_order failure_order) noexcept;
```


Предусловия

Параметр `failure_order` не должен быть равен `std::memory_order_release` или `std::memory_order_acq_rel`.

Результат

Атомарно сравнивает ожидаемое значение `expected` со значением, сохраненным в `*this`, используя для этого поразрядное сравнение, и сохраняет `new_value` в `*this`, если значения равны. В противном случае обновляет `expected` считанным значением.

Возвращает

`true`, если имевшееся значение `*this` было равно значению `expected`, или `false` в противном случае.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это переопределение с тремя параметрами эквивалентно переопределению с четырьмя параметрами, где `success_order==order` и `failure_order==order`, за исключением того, что если `order` имеет значение `std::memory_order_acq_rel`, то `failure_order` имеет значение `std::memory_order_acquire`, а если `order` имеет значение `std::memory_order_release`, то `failure_order` имеет значение `std::memory_order_relaxed`.

ПРИМЕЧАНИЕ

Если при порядке доступа к памяти `success_order` результатом является `true`, то для места в памяти, содержащего `*this`, это будет атомарной операцией чтения — изменения — записи; в ином случае при порядке доступа к памяти `failure_order` для места в памяти, содержащего `*this`, это атомарная операция загрузки.

Некомпонентная функция `std::atomic_compare_exchange_strong`

Атомарно сравнивает значение с ожидаемым значением и сохраняет новое значение, если они равны друг другу. Если значения не равны, обновляет ожидаемое значение считанным значением.

Объявление

```
template<typename BaseType>
bool atomic_compare_exchange_strong(
    volatile atomic<BaseType>* p, BaseType * old_value, BaseType new_value)
    noexcept;
template<typename BaseType>
bool atomic_compare_exchange_strong(
    atomic<BaseType>* p, BaseType * old_value, BaseType new_value) noexcept;
```

Результат

```
return p->compare_exchange_strong(*old_value, new_value);
```

Некомпонентная функция `std::atomic_compare_exchange_strong_explicit`

Атомарно сравнивает значение с ожидаемым значением и сохраняет новое значение, если они равны друг другу. Если значения не равны, обновляет ожидаемое значение считанным значением.

Объявление

```
template<typename BaseType>
bool atomic_compare_exchange_strong_explicit(
    volatile atomic<BaseType>* p, BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_strong_explicit(
    atomic<BaseType>* p, BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
```

Результат

```
return p->compare_exchange_strong(
    *old_value, new_value, success_order, failure_order) noexcept;
```

Компонентная функция `std::atomic::compare_exchange_weak`

Атомарно сравнивает значение с ожидаемым значением и сохраняет новое значение, если они равны друг другу и обновление можно выполнить атомарно. Если значения не равны или обновление нельзя выполнить атомарно, обновляет ожидаемое значение считанным значением.

Объявление

```
bool compare_exchange_weak(
    BaseType& expected, BaseType new_value,
    memory_order order = std::memory_order_seq_cst) volatile noexcept;
bool compare_exchange_weak(
    BaseType& expected, BaseType new_value,
    memory_order order = std::memory_order_seq_cst) noexcept;
bool compare_exchange_weak(
    BaseType& expected, BaseType new_value,
    memory_order success_order, memory_order failure_order)
    volatile noexcept;
bool compare_exchange_weak(
    BaseType& expected, BaseType new_value,
    memory_order success_order, memory_order failure_order) noexcept;
```

Предусловия

Параметр `failure_order` не должен быть равен `std::memory_order_release` или `std::memory_order_acq_rel`.

Результат

Атомарно поразрядно сравнивает ожидаемое значение со значением, сохраненным в **this*, и сохраняет в **this* значение *new_value*, если сравниваемые значения равны друг другу. Если значения не равны или обновление нельзя выполнить атомарно, обновляет ожидаемое значение считанным значением.

Возвращает

true, если имеющееся значение **this* было равно значению *expected* и *new_value* было успешно сохранено в **this*, или *false* в противном случае.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это переопределение с тремя параметрами эквивалентно переопределению с четырьмя параметрами, где *success_order==order* и *failure_order==order*, за исключением того, что если *order* имеет значение *std::memory_order_acq_rel*, то *failure_order* имеет значение *std::memory_order_acquire*, а если *order* имеет значение *std::memory_order_release*, то *failure_order* имеет значение *std::memory_order_relaxed*.

ПРИМЕЧАНИЕ

Если при порядке доступа к памяти *success_order* результатом является *true*, то для места в памяти, содержащего **this*, это будет атомарной операцией чтения — изменения — записи; в противном случае при порядке доступа к памяти *failure_order* для места в памяти, содержащего **this*, это атомарная операция загрузки.

Некомпонентная функция *std::atomic_compare_exchange_weak*

Атомарно сравнивает значение с ожидаемым значением и сохраняет новое значение, если они равны друг другу и обновление можно выполнить атомарно. Если значения не равны или обновление нельзя выполнить атомарно, обновляет ожидаемое значение считанным значением.

Объявление

```
template<typename BaseType>
bool atomic_compare_exchange_weak(
    volatile atomic<BaseType>* p, BaseType * old_value, BaseType new_value)
    noexcept;
template<typename BaseType>
bool atomic_compare_exchange_weak(
    atomic<BaseType>* p, BaseType * old_value, BaseType new_value) noexcept;
```

Результат

```
return p->compare_exchange_weak(*old_value, new_value);
```

Некомпонентная функция `std::atomic_compare_exchange_weak_explicit`

Атомарно сравнивает значение с ожидаемым значением и сохраняет новое значение, если они равны друг другу и обновление можно выполнить атомарно. Если значения не равны или обновление нельзя выполнить атомарно, обновляет ожидаемое значение считанным значением.

Объявление

```
template<typename BaseType>
bool atomic_compare_exchange_weak_explicit(
    volatile atomic<BaseType>* p, BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_weak_explicit(
    atomic<BaseType>* p, BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
```

Результат

```
return p->compare_exchange_weak(
    *old_value, new_value, success_order, failure_order);
```

Г.3.9. Специализации шаблона `std::atomic`

Специализации шаблона класса `std::atomic` предоставляются для целочисленных и указательных типов. В дополнение к операциям, предоставляемым основным шаблоном, для целочисленных типов эти специализации предоставляют атомарные операции сложения, вычитания и поразрядные операции, а для типов указателей — атомарные арифметические операции над указателями.

Специализации предоставляются для следующих целочисленных типов:

```
std::atomic<bool>
std::atomic<char>
std::atomic<signed char>
std::atomic<unsigned char>
std::atomic<short>
std::atomic<unsigned short>
std::atomic<int>
std::atomic<unsigned>
std::atomic<long>
std::atomic<unsigned long>
std::atomic<long long>
std::atomic<unsigned long long>
std::atomic<wchar_t>
std::atomic<char16_t>
std::atomic<char32_t>
```

И `std::atomic<T*>` для всех типов `T`.

Г.3.10. Специализации `std::atomic<integral-type>`

Специализации `std::atomic<integral-type>` шаблона класса `std::atomic` предоставляют атомарный целочисленный тип данных для каждого фундаментального целочисленного типа с полным набором операций.

К этим специализациям шаблона класса `std::atomic<>` применимы следующие описания:

```
std::atomic<char>
std::atomic<signed char>
std::atomic<unsigned char>
std::atomic<short>
std::atomic<unsigned short>
std::atomic<int>
std::atomic<unsigned>
std::atomic<long>
std::atomic<unsigned long>
std::atomic<long long>
std::atomic<unsigned long long>
std::atomic<wchar_t>
std::atomic<char16_t>
std::atomic<char32_t>
```

Экземпляры этих специализаций не являются `CopyConstructible` или `CopyAssignable`, поскольку соответствующие им операции не могут выполняться в рамках единой атомарной операции.

Определение класса

```
template<>
struct atomic<integral-type>
{
    atomic() noexcept = default;
    constexpr atomic(integral-type) noexcept;
    bool operator=(integral-type) volatile noexcept;

    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;

    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(integral-type, memory_order = memory_order_seq_cst)
        volatile noexcept;
    void store(integral-type, memory_order = memory_order_seq_cst) noexcept;
    integral-type load(memory_order = memory_order_seq_cst)
        const volatile noexcept;
    integral-type load(memory_order = memory_order_seq_cst) const noexcept;
    integral-type exchange(
        integral-type, memory_order = memory_order_seq_cst)
        volatile noexcept;
    integral-type exchange(
```

```

    integral-type,memory_order = memory_order_seq_cst) noexcept;
bool compare_exchange_strong(
    integral-type & old_value,integral-type new_value,
    memory_order order = memory_order_seq_cst) volatile noexcept;
bool compare_exchange_strong(
    integral-type & old_value,integral-type new_value,
    memory_order order = memory_order_seq_cst) noexcept;
bool compare_exchange_strong(
    integral-type & old_value,integral-type new_value,
    memory_order success_order,memory_order failure_order)
    volatile noexcept;
bool compare_exchange_strong(
    integral-type & old_value,integral-type new_value,
    memory_order success_order,memory_order failure_order) noexcept;
bool compare_exchange_weak(
    integral-type & old_value,integral-type new_value,
    memory_order order = memory_order_seq_cst) volatile noexcept;
bool compare_exchange_weak(
    integral-type & old_value,integral-type new_value,
    memory_order order = memory_order_seq_cst) noexcept;
bool compare_exchange_weak(
    integral-type & old_value,integral-type new_value,
    memory_order success_order,memory_order failure_order)
    volatile noexcept;
bool compare_exchange_weak(
    integral-type & old_value,integral-type new_value,
    memory_order success_order,memory_order failure_order) noexcept;

operator integral-type() const volatile noexcept;
operator integral-type() const noexcept;

integral-type fetch_add(
    integral-type,memory_order = memory_order_seq_cst)
    volatile noexcept;
integral-type fetch_add(
    integral-type,memory_order = memory_order_seq_cst) noexcept;
integral-type fetch_sub(
    integral-type,memory_order = memory_order_seq_cst)
    volatile noexcept;
integral-type fetch_sub(
    integral-type,memory_order = memory_order_seq_cst) noexcept;
integral-type fetch_and(
    integral-type,memory_order = memory_order_seq_cst)
    volatile noexcept;
integral-type fetch_and(
    integral-type,memory_order = memory_order_seq_cst) noexcept;
integral-type fetch_or(
    integral-type,memory_order = memory_order_seq_cst)
    volatile noexcept;
integral-type fetch_or(
    integral-type,memory_order = memory_order_seq_cst) noexcept;
integral-type fetch_xor(

```



```

    integral-type, memory_order = memory_order_seq_cst)
    volatile noexcept;
    integral-type fetch_xor(
        integral-type, memory_order = memory_order_seq_cst) noexcept;

    integral-type operator++() volatile noexcept;
    integral-type operator++() noexcept;
    integral-type operator++(int) volatile noexcept;
    integral-type operator++(int) noexcept;
    integral-type operator--() volatile noexcept;
    integral-type operator--() noexcept;
    integral-type operator--(int) volatile noexcept;
    integral-type operator--(int) noexcept;

    integral-type operator+=(integral-type) volatile noexcept;
    integral-type operator+=(integral-type) noexcept;
    integral-type operator-=(integral-type) volatile noexcept;
    integral-type operator-=(integral-type) noexcept;
    integral-type operator&=(integral-type) volatile noexcept;
    integral-type operator&=(integral-type) noexcept;
    integral-type operator|=(integral-type) volatile noexcept;
    integral-type operator|=(integral-type) noexcept;
    integral-type operator^=(integral-type) volatile noexcept;
    integral-type operator^=(integral-type) noexcept;
};
bool atomic_is_lock_free(volatile const atomic<integral-type>*) noexcept;
bool atomic_is_lock_free(const atomic<integral-type>*) noexcept;
void atomic_init(volatile atomic<integral-type>*, integral-type) noexcept;
void atomic_init(atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_exchange(
    volatile atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_exchange(
    atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_exchange_explicit(
    volatile atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_exchange_explicit(
    atomic<integral-type>*, integral-type, memory_order) noexcept;
void atomic_store(volatile atomic<integral-type>*, integral-type) noexcept;
void atomic_store(atomic<integral-type>*, integral-type) noexcept;
void atomic_store_explicit(
    volatile atomic<integral-type>*, integral-type, memory_order) noexcept;
void atomic_store_explicit(
    atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_load(volatile const atomic<integral-type>*) noexcept;
integral-type atomic_load(const atomic<integral-type>*) noexcept;
integral-type atomic_load_explicit(
    volatile const atomic<integral-type>*, memory_order) noexcept;
integral-type atomic_load_explicit(
    const atomic<integral-type>*, memory_order) noexcept;
bool atomic_compare_exchange_strong(
    volatile atomic<integral-type>*,
    integral-type * old_value, integral-type new_value) noexcept;

```



```

bool atomic_compare_exchange_strong(
    atomic<integral-type>*,
    integral-type * old_value, integral-type new_value) noexcept;
bool atomic_compare_exchange_strong_explicit(
    volatile atomic<integral-type>*,
    integral-type * old_value, integral-type new_value,
    memory_order success_order, memory_order failure_order) noexcept;
bool atomic_compare_exchange_strong_explicit(
    atomic<integral-type>*,
    integral-type * old_value, integral-type new_value,
    memory_order success_order, memory_order failure_order) noexcept;
bool atomic_compare_exchange_weak(
    volatile atomic<integral-type>*,
    integral-type * old_value, integral-type new_value) noexcept;
bool atomic_compare_exchange_weak(
    atomic<integral-type>*,
    integral-type * old_value, integral-type new_value) noexcept;
bool atomic_compare_exchange_weak_explicit(
    volatile atomic<integral-type>*,
    integral-type * old_value, integral-type new_value,
    memory_order success_order, memory_order failure_order) noexcept;
bool atomic_compare_exchange_weak_explicit(
    atomic<integral-type>*,
    integral-type * old_value, integral-type new_value,
    memory_order success_order, memory_order failure_order) noexcept;

integral-type atomic_fetch_add(
    volatile atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_add(
    atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_add_explicit(
    volatile atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_add_explicit(
    atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_sub(
    volatile atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_sub(
    atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_sub_explicit(
    volatile atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_sub_explicit(
    atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_and(
    volatile atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_and(
    atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_and_explicit(
    volatile atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_and_explicit(
    atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_or(
    volatile atomic<integral-type>*, integral-type) noexcept;

```

```

integral-type atomic_fetch_or(
    atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_or_explicit(
    volatile atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_or_explicit(
    atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_xor(
    volatile atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_xor(
    atomic<integral-type>*, integral-type) noexcept;
integral-type atomic_fetch_xor_explicit(
    volatile atomic<integral-type>*, integral-type, memory_order) noexcept;
integral-type atomic_fetch_xor_explicit(
    atomic<integral-type>*, integral-type, memory_order) noexcept;

```

Такую же семантику имеют и те операции, которые также предоставляются основным шаблоном (см. подраздел Г.3.8).

Компонентная функция `std::atomic<integral-type>::fetch_add`

Атомарно загружает значение и заменяет его суммой этого значения и предоставленного значения `i`.

Объявление

```

integral-type fetch_add(
    integral-type i, memory_order order = memory_order_seq_cst)
    volatile noexcept;
integral-type fetch_add(
    integral-type i, memory_order order = memory_order_seq_cst) noexcept;

```

Результат

Атомарно извлекает имеющееся значение `*this` и сохраняет в `*this` сумму `old-value + i`.

Возвращает

Значение `*this`, непосредственно предшествующее сохранению.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Для места в памяти, содержащего `*this`, эта функция является атомарной операцией чтения — изменения — записи.

Некомпонентная функция `std::atomic_fetch_add`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его значением плюс предоставленное значение `i`.

Объявление

```
integral-type atomic_fetch_add(
    volatile atomic<integral-type>* p, integral-type i) noexcept;
integral-type atomic_fetch_add(
    atomic<integral-type>* p, integral-type i) noexcept;
```

Результат

```
return p->fetch_add(i);
```

Некомпонентная функция std::atomic_fetch_add_explicit

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его значением плюс предоставленное значение `i`.

Объявление

```
integral-type atomic_fetch_add_explicit(
    volatile atomic<integral-type>* p, integral-type i,
    memory_order order) noexcept;
integral-type atomic_fetch_add_explicit(
    atomic<integral-type>* p, integral-type i, memory_order order)
    noexcept;
```

Результат

```
return p->fetch_add(i,order);
```

Компонентная функция std::atomic<integral-type>::fetch_sub

Атомарно загружает значение и заменяет его разностью этого значения и предоставленного значения `i`.

Объявление

```
integral-type fetch_sub(
    integral-type i, memory_order order = memory_order_seq_cst)
    volatile noexcept;
integral-type fetch_sub(
    integral-type i, memory_order order = memory_order_seq_cst) noexcept;
```

Результат

Атомарно извлекает имеющееся значение `*this` и сохраняет в `*this` разность `old-value - i`.

Возвращает

Значение `*this`, непосредственно предшествующее сохранению.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Для места в памяти, содержащего `*this`, эта функция является атомарной операцией чтения — изменения — записи.

Некомпонентная функция `std::atomic_fetch_sub`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его значением минус предоставленное значение `i`.

Объявление

```
integral-type atomic_fetch_sub(
    volatile atomic<integral-type>* p, integral-type i) noexcept;
integral-type atomic_fetch_sub(
    atomic<integral-type>* p, integral-type i) noexcept;
```

Результат

```
return p->fetch_sub(i);
```

Некомпонентная функция `std::atomic_fetch_sub_explicit`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его значением с вычетом предоставленного значения `i`.

Объявление

```
integral-type atomic_fetch_sub_explicit(
    volatile atomic<integral-type>* p, integral-type i,
    memory_order order) noexcept;
integral-type atomic_fetch_sub_explicit(
    atomic<integral-type>* p, integral-type i, memory_order order)
    noexcept;
```

Результат

```
return p->fetch_sub(i,order);
```

Компонентная функция `std::atomic<integral-type>::fetch_and`

Атомарно загружает значение и заменяет его результатом операции «поразрядное И» между этим значением и аргументом `i`.

Объявление

```
integral-type fetch_and(
    integral-type i, memory_order order = memory_order_seq_cst)
    volatile noexcept;
integral-type fetch_and(
    integral-type i, memory_order order = memory_order_seq_cst) noexcept;
```

Результат

Атомарно извлекает имеющееся значение `*this` и сохраняет в `*this` результат вычисления `old-value & i`.

Возвращает

Значение `*this`, непосредственно предшествующее сохранению.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Для места в памяти, содержащего **this*, эта функция является атомарной операцией чтения — изменения — записи.

Некомпонентная функция `std::atomic_fetch_and`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его результатом операции «поразрядное И» между этим значением и аргументом *i*.

Объявление

```
integral-type atomic_fetch_and(
    volatile atomic<integral-type>* p, integral-type i) noexcept;
integral-type atomic_fetch_and(
    atomic<integral-type>* p, integral-type i) noexcept;
```

Результат

```
return p->fetch_and(i);
```

Некомпонентная функция `std::atomic_fetch_and_explicit`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его результатом операции «поразрядное И» между этим значением и аргументом *i*.

Объявление

```
integral-type atomic_fetch_and_explicit(
    volatile atomic<integral-type>* p, integral-type i,
    memory_order order) noexcept;
integral-type atomic_fetch_and_explicit(
    atomic<integral-type>* p, integral-type i, memory_order order)
    noexcept;
```

Результат

```
return p->fetch_and(i,order);
```

Компонентная функция**`std::atomic<integral-type>::fetch_or`**

Атомарно загружает значение и заменяет его результатом операции «поразрядное ИЛИ» между этим значением и аргументом *i*.

Объявление

```
integral-type fetch_or(
    integral-type i, memory_order order = memory_order_seq_cst)
    volatile noexcept;
integral-type fetch_or(
    integral-type i, memory_order order = memory_order_seq_cst) noexcept;
```

Результат

Атомарно извлекает имеющееся значение **this* и сохраняет в **this* результат вычисления `old-value | i`.

Возвращает

Значение `*this`, непосредственно предшествующее сохранению.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Для места в памяти, содержащего `*this`, эта функция является атомарной операцией чтения — изменения — записи.

Некомпонентная функция `std::atomic_fetch_or`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его результатом операции «поразрядное ИЛИ» между этим значением и аргументом `i`.

Объявление

```
integral-type atomic_fetch_or(  
    volatile atomic<integral-type>* p, integral-type i) noexcept;  
integral-type atomic_fetch_or(  
    atomic<integral-type>* p, integral-type i) noexcept;
```

Результат

```
return p->fetch_or(i);
```

Некомпонентная функция `std::atomic_fetch_or_explicit`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его результатом операции «поразрядное ИЛИ» между этим значением и аргументом `i`.

Объявление

```
integral-type atomic_fetch_or_explicit(  
    volatile atomic<integral-type>* p, integral-type i,  
    memory_order order) noexcept;  
integral-type atomic_fetch_or_explicit(  
    atomic<integral-type>* p, integral-type i, memory_order order)  
    noexcept;
```

Результат

```
return p->fetch_or(i,order);
```

Компонентная функция `std::atomic<integral-type>::fetch_xor`

Атомарно загружает значение и заменяет его результатом операции «поразрядное исключающее ИЛИ» между этим значением и аргументом `i`.

Объявление

```
integral-type fetch_xor(  
    integral-type i, memory_order order = memory_order_seq_cst)  
    volatile noexcept;  
integral-type fetch_xor(  
    integral-type i, memory_order order = memory_order_seq_cst) noexcept;
```

Результат

Атомарно извлекает имеющееся значение **this* и сохраняет в **this* результат вычисления *old-value* ^ *i*.

Возвращает

Значение **this*, непосредственно предшествующее сохранению.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Для места в памяти, содержащего **this*, эта функция является атомарной операцией чтения — изменения — записи.

Некомпонентная функция `std::atomic_fetch_xor`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его результатом операции «поразрядное исключающее ИЛИ» между этим значением и аргументом *i*.

Объявление

```
integral-type atomic_fetch_xor(
    volatile atomic<integral-type>* p, integral-type i) noexcept;
integral-type atomic_fetch_xor(
    atomic<integral-type>* p, integral-type i) noexcept;
```

Результат

```
return p->fetch_xor(i);
```

Некомпонентная функция `std::atomic_fetch_xor_explicit`

Атомарно считывает значение из экземпляра `atomic<integral-type>` и заменяет его результатом операции «поразрядное исключающее ИЛИ» между этим значением и аргументом *i*.

Объявление

```
integral-type atomic_fetch_xor_explicit(
    volatile atomic<integral-type>* p, integral-type i,
    memory_order order) noexcept;
integral-type atomic_fetch_xor_explicit(
    atomic<integral-type>* p, integral-type i, memory_order order)
    noexcept;
```

Результат

```
return p->fetch_xor(i,order);
```

Оператор прединкремента `std::atomic<integral-type>::operator++`

Атомарно увеличивает значение, сохраненное в **this*, на единицу и возвращает новое значение.

Объявление

```
integral-type operator++() volatile noexcept;  
integral-type operator++() noexcept;
```

Результат

```
return this->fetch_add(1) + 1;
```

Оператор постинкремента**std::atomic<integral-type>::operator++**

Атомарно увеличивает на единицу значение, сохраненное в **this*, и возвращает старое значение.

Объявление

```
integral-type operator++(int) volatile noexcept;  
integral-type operator++(int) noexcept;
```

Результат

```
return this->fetch_add(1);
```

Оператор предекремента**std::atomic<integral-type>::operator--**

Атомарно уменьшает на единицу значение, сохраненное в **this*, и возвращает новое значение.

Объявление

```
integral-type operator--() volatile noexcept;  
integral-type operator--() noexcept;
```

Результат

```
return this->fetch_sub(1) - 1;
```

Оператор постдекремента**std::atomic<integral-type>::operator--**

Атомарно уменьшает на единицу значение, сохраненное в **this*, и возвращает старое значение.

Объявление

```
integral-type operator--(int) volatile noexcept;  
integral-type operator--(int) noexcept;
```

Результат

```
return this->fetch_sub(1);
```

Составной оператор**присваивания std::atomic<integral-type>::operator+=**

Атомарно складывает предоставленное значение со значением, сохраненным в **this*, и возвращает новое значение.

Объявление

```
integral-type operator+=(integral-type i) volatile noexcept;
integral-type operator+=(integral-type i) noexcept;
```

Результат

```
return this->fetch_add(i) + i;
```

Составной оператор**присваивания `std::atomic<integral-type>::operator--`**

Атомарно вычитает предоставленное значение из значения, сохраненного в **this*, и возвращает новое значение.

Объявление

```
integral-type operator--(integral-type i) volatile noexcept;
integral-type operator--(integral-type i) noexcept;
```

Результат

```
return this->fetch_sub(i, std::memory_order_seq_cst) - i;
```

Составной оператор**присваивания `std::atomic<integral-type>::operator&=`**

Атомарно заменяет значение, сохраненное в **this*, результатом операции «поразрядное И» между предоставленным значением и значением, сохраненным в **this*, и возвращает новое значение.

Объявление

```
integral-type operator&=(integral-type i) volatile noexcept;
integral-type operator&=(integral-type i) noexcept;
```

Результат

```
return this->fetch_and(i) & i;
```

Составной оператор**присваивания `std::atomic<integral-type>::operator|=`**

Атомарно заменяет значение, сохраненное в **this*, результатом операции «поразрядное ИЛИ» между предоставленным значением и значением, сохраненным в **this*, и возвращает новое значение.

Объявление

```
integral-type operator|=(integral-type i) volatile noexcept;
integral-type operator|=(integral-type i) noexcept;
```

Результат

```
return this->fetch_or(i, std::memory_order_seq_cst) | i;
```

Составной оператор присваивания `std::atomic<integral-type>::operator^=`

Атомарно заменяет значение, сохраненное в `*this`, результатом операции «поразрядное исключающее ИЛИ» между предоставленным значением и значением, сохраненным в `*this`, и возвращает новое значение.

Объявление

```
integral-type operator^=(integral-type i) volatile noexcept;
integral-type operator^=(integral-type i) noexcept;
```

Результат

```
return this->fetch_xor(i, std::memory_order_seq_cst) ^ i;
```

Частичная специализация `std::atomic<T*>`

Частичная специализация `std::atomic<T*>` шаблона класса `std::atomic` предоставляет атомарный тип данных для каждого типа указателя с полным набором операций.

Экземпляры `std::atomic<T*>` не являются `CopyConstructible` или `CopyAssignable`, поскольку соответствующие им операции не могут выполняться в рамках единой атомарной операции.

Определение класса

```
template<typename T>
struct atomic<T*>
{
    atomic() noexcept = default;
    constexpr atomic(T*) noexcept;
    bool operator=(T*) volatile;
    bool operator=(T*);

    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;

    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(T*, memory_order = memory_order_seq_cst) volatile noexcept;
    void store(T*, memory_order = memory_order_seq_cst) noexcept;
    T* load(memory_order = memory_order_seq_cst) const volatile noexcept;
    T* load(memory_order = memory_order_seq_cst) const noexcept;
    T* exchange(T*, memory_order = memory_order_seq_cst) volatile noexcept;
    T* exchange(T*, memory_order = memory_order_seq_cst) noexcept;

    bool compare_exchange_strong(
        T* & old_value, T* new_value,
        memory_order order = memory_order_seq_cst) volatile noexcept;
    bool compare_exchange_strong(
        T* & old_value, T* new_value,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_strong(
        T* & old_value, T* new_value,
```

```

        memory_order success_order, memory_order failure_order)
        volatile noexcept;
bool compare_exchange_strong(
    T* & old_value, T* new_value,
    memory_order success_order, memory_order failure_order) noexcept;
bool compare_exchange_weak(
    T* & old_value, T* new_value,
    memory_order order = memory_order_seq_cst) volatile noexcept;
bool compare_exchange_weak(
    T* & old_value, T* new_value,
    memory_order order = memory_order_seq_cst) noexcept;
bool compare_exchange_weak(
    T* & old_value, T* new_value,
    memory_order success_order, memory_order failure_order)
    volatile noexcept;
bool compare_exchange_weak(
    T* & old_value, T* new_value,
    memory_order success_order, memory_order failure_order) noexcept;

operator T*() const volatile noexcept;
operator T*() const noexcept;

T* fetch_add(
    ptrdiff_t, memory_order = memory_order_seq_cst) volatile noexcept;
T* fetch_add(
    ptrdiff_t, memory_order = memory_order_seq_cst) noexcept;
T* fetch_sub(
    ptrdiff_t, memory_order = memory_order_seq_cst) volatile noexcept;
T* fetch_sub(
    ptrdiff_t, memory_order = memory_order_seq_cst) noexcept;

T* operator++() volatile noexcept;
T* operator++() noexcept;
T* operator++(int) volatile noexcept;
T* operator++(int) noexcept;
T* operator--() volatile noexcept;
T* operator--() noexcept;
T* operator--(int) volatile noexcept;
T* operator--(int) noexcept;
T* operator+=(ptrdiff_t) volatile noexcept;
T* operator+=(ptrdiff_t) noexcept;
T* operator-=(ptrdiff_t) volatile noexcept;
T* operator-=(ptrdiff_t) noexcept;
};

bool atomic_is_lock_free(volatile const atomic<T*>*) noexcept;
bool atomic_is_lock_free(const atomic<T*>*) noexcept;
void atomic_init(volatile atomic<T*>*, T*) noexcept;
void atomic_init(atomic<T*>*, T*) noexcept;
T* atomic_exchange(volatile atomic<T*>*, T*) noexcept;
T* atomic_exchange(atomic<T*>*, T*) noexcept;
T* atomic_exchange_explicit(volatile atomic<T*>*, T*, memory_order)
    noexcept;
T* atomic_exchange_explicit(atomic<T*>*, T*, memory_order) noexcept;
void atomic_store(volatile atomic<T*>*, T*) noexcept;
void atomic_store(atomic<T*>*, T*) noexcept;
void atomic_store_explicit(volatile atomic<T*>*, T*, memory_order)
    noexcept;
void atomic_store_explicit(atomic<T*>*, T*, memory_order) noexcept;

```

```

void atomic_store_explicit(volatile atomic<T*>*, T*, memory_order) noexcept;
void atomic_store_explicit(atomic<T*>*, T*, memory_order) noexcept;
T* atomic_load(volatile const atomic<T*>*) noexcept;
T* atomic_load(const atomic<T*>*) noexcept;
T* atomic_load_explicit(volatile const atomic<T*>*, memory_order) noexcept;
T* atomic_load_explicit(const atomic<T*>*, memory_order) noexcept;
bool atomic_compare_exchange_strong(
    volatile atomic<T*>*, T* * old_value, T* new_value) noexcept;
bool atomic_compare_exchange_strong(
    volatile atomic<T*>*, T* * old_value, T* new_value) noexcept;
bool atomic_compare_exchange_strong_explicit(
    atomic<T*>*, T* * old_value, T* new_value,
    memory_order success_order, memory_order failure_order) noexcept;
bool atomic_compare_exchange_strong_explicit(
    atomic<T*>*, T* * old_value, T* new_value,
    memory_order success_order, memory_order failure_order) noexcept;
bool atomic_compare_exchange_weak(
    volatile atomic<T*>*, T* * old_value, T* new_value) noexcept;
bool atomic_compare_exchange_weak(
    atomic<T*>*, T* * old_value, T* new_value) noexcept;
bool atomic_compare_exchange_weak_explicit(
    volatile atomic<T*>*, T* * old_value, T* new_value,
    memory_order success_order, memory_order failure_order) noexcept;
bool atomic_compare_exchange_weak_explicit(
    atomic<T*>*, T* * old_value, T* new_value,
    memory_order success_order, memory_order failure_order) noexcept;

T* atomic_fetch_add(volatile atomic<T*>*, ptrdiff_t) noexcept;
T* atomic_fetch_add(atomic<T*>*, ptrdiff_t) noexcept;
T* atomic_fetch_add_explicit(
    volatile atomic<T*>*, ptrdiff_t, memory_order) noexcept;
T* atomic_fetch_add_explicit(
    atomic<T*>*, ptrdiff_t, memory_order) noexcept;
T* atomic_fetch_sub(volatile atomic<T*>*, ptrdiff_t) noexcept;
T* atomic_fetch_sub(atomic<T*>*, ptrdiff_t) noexcept;
T* atomic_fetch_sub_explicit(
    volatile atomic<T*>*, ptrdiff_t, memory_order) noexcept;
T* atomic_fetch_sub_explicit(
    atomic<T*>*, ptrdiff_t, memory_order) noexcept;

```

Такую же семантику имеют и те операции, которые также предоставляются основным шаблоном (см. подраздел Г.3.8).

Компонентная функция `std::atomic<T*>::fetch_add`

Атомарно загружает значение и заменяет его суммой этого значения и предоставленного значения `i` с применением стандартных правил арифметики указателей, возвращает старое значение.

Объявление

```

T* fetch_add(
    ptrdiff_t i, memory_order order = memory_order_seq_cst)
    volatile noexcept;
T* fetch_add(
    ptrdiff_t i, memory_order order = memory_order_seq_cst) noexcept;

```

Результат

Атомарно извлекает имеющееся значение `*this` и сохраняет в `*this` результат вычисления `old-value + i`.

Возвращает

Значение `*this`, непосредственно предшествующее сохранению.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Для места в памяти, содержащего `*this`, эта функция является атомарной операцией чтения — изменения — записи.

Некомпонентная функция `std::atomic_fetch_add`

Атомарно считывает значение из экземпляра `atomic<T*>` и заменяет его этим значением плюс предоставленное значение `i` с применением стандартных правил арифметики указателей.

Объявление

```
T* atomic_fetch_add(volatile atomic<T*>* p, ptrdiff_t i) noexcept;
T* atomic_fetch_add(atomic<T*>* p, ptrdiff_t i) noexcept;
```

Результат

```
return p->fetch_add(i);
```

Некомпонентная функция `std::atomic_fetch_add_explicit`

Атомарно считывает значение из экземпляра `atomic<T*>` и заменяет его этим значением плюс предоставленное значение `i` с применением стандартных правил арифметики указателей.

Объявление

```
T* atomic_fetch_add_explicit(
    volatile atomic<T*>* p, ptrdiff_t i, memory_order order) noexcept;
T* atomic_fetch_add_explicit(
    atomic<T*>* p, ptrdiff_t i, memory_order order) noexcept;
```

Результат

```
return p->fetch_add(i, order);
```

Компонентная функция `std::atomic<T*>::fetch_sub`

Атомарно загружает значение и заменяет его этим значением минус предоставленное значение `i` с применением стандартных правил арифметики указателей, возвращает старое значение.

Объявление

```
T* fetch_sub(
    ptrdiff_t i, memory_order order = memory_order_seq_cst)
    volatile noexcept;
T* fetch_sub(
    ptrdiff_t i, memory_order order = memory_order_seq_cst) noexcept;
```

Результат

Атомарно извлекает имеющееся значение **this* и сохраняет в **this* результат вычисления *old-value - i*.

Возвращает

Значение **this*, непосредственно предшествующее сохранению.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Для места в памяти, содержащего **this*, эта функция является атомарной операцией чтения — изменения — записи.

Некомпонентная функция `std::atomic_fetch_sub`

Атомарно считывает значение из экземпляра `atomic<T*>` и заменяет его этим значением минус предоставленное значение *i* с применением стандартных правил арифметики указателей.

Объявление

```
T* atomic_fetch_sub(volatile atomic<T*>* p, ptrdiff_t i) noexcept;
T* atomic_fetch_sub(atomic<T*>* p, ptrdiff_t i) noexcept;
```

Результат

```
return p->fetch_sub(i);
```

Некомпонентная функция `std::atomic_fetch_sub_explicit`

Атомарно считывает значение из экземпляра `atomic<T*>` и заменяет его этим значением минус предоставленное значение *i* с применением стандартных правил арифметики указателей.

Объявление

```
T* atomic_fetch_sub_explicit(
    volatile atomic<T*>* p, ptrdiff_t i, memory_order order) noexcept;
T* atomic_fetch_sub_explicit(
    atomic<T*>* p, ptrdiff_t i, memory_order order) noexcept;
```

Результат

```
return p->fetch_sub(i, order);
```


Оператор прединкремента `std::atomic<T*>::operator++`

Атомарно увеличивает на единицу значение, сохраненное в `*this`, применяя стандартные правила арифметики указателей, и возвращает новое значение.

Объявление

```
T* operator++() volatile noexcept;
T* operator++() noexcept;
```

Результат

```
return this->fetch_add(1) + 1;
```

Оператор постинкремента `std::atomic<T*>::operator++`

Атомарно увеличивает на единицу значение, сохраненное в `*this`, и возвращает старое значение.

Объявление

```
T* operator++(int) volatile noexcept;
T* operator++(int) noexcept;
```

Результат

```
return this->fetch_add(1);
```

Оператор преддекремента `std::atomic<T*>::operator--`

Атомарно уменьшает на единицу значение, сохраненное в `*this`, применяя стандартные правила арифметики указателей, и возвращает новое значение.

Объявление

```
T* operator--() volatile noexcept;
T* operator--() noexcept;
```

Результат

```
return this->fetch_sub(1) - 1;
```

Оператор постдекремента `std::atomic<T*>::operator--`

Атомарно уменьшает на единицу значение, сохраненное в `*this`, применяя стандартные правила арифметики указателей, и возвращает старое значение.

Объявление

```
T* operator--(int) volatile noexcept;
T* operator--(int) noexcept;
```

Результат

```
return this->fetch_sub(1);
```

Составной оператор присваивания `std::atomic<T*>::operator+=`

Атомарно прибавляет предоставленное значение к значению, сохраненному в `*this`, применяя стандартные правила арифметики указателей, и возвращает новое значение.